

LAPORAN PRAKTIKUM
IMPLEMENTASI ARTIFICIAL NEURAL NETWORK (ANN) FROM SCRATCH
MENGGUNAKAN NUMPY DAN FRAMEWORK KERAS PADA KLASIFIKASI DRY
BEAN DATASET



Disusun oleh:

Barep Purwo Agosto (2303030003)

Nabila Rahayu (2303030015)

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS DARUNNAJAH

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi Artificial Intelligence (AI) telah membawa perubahan besar dalam berbagai bidang, seperti kesehatan, pertanian, industri manufaktur, sistem keamanan, transportasi, hingga sektor bisnis. Salah satu cabang utama dari Artificial Intelligence adalah Machine Learning, yaitu teknik yang memungkinkan komputer mempelajari pola dari data tanpa diprogram secara eksplisit. Di dalam Machine Learning, metode yang paling berkembang pesat adalah Deep Learning yang menggunakan Artificial Neural Network (ANN) sebagai dasar utama proses pembelajaran.

Artificial Neural Network merupakan model komputasi yang terinspirasi dari cara kerja jaringan saraf biologis pada otak manusia. ANN mampu mempelajari hubungan kompleks antara data masukan dan keluaran melalui proses pembelajaran yang melibatkan pembaruan bobot (weight) secara berulang menggunakan algoritma optimisasi. Dengan kemampuan tersebut, ANN banyak digunakan pada berbagai permasalahan seperti klasifikasi citra, pengenalan suara, deteksi objek, hingga prediksi data.

Saat ini implementasi Neural Network umumnya dilakukan menggunakan framework modern seperti TensorFlow, Keras, maupun PyTorch. Framework tersebut menyediakan berbagai fungsi siap pakai yang memudahkan proses pembangunan model sehingga pengembang tidak perlu lagi membuat algoritma pembelajaran dari awal. Meskipun demikian, kemudahan tersebut sering kali menyebabkan pengguna kurang memahami mekanisme internal Neural Network, terutama mengenai proses Forward Propagation, Backpropagation, perhitungan fungsi loss, serta pembaruan bobot menggunakan algoritma Gradient Descent.

Oleh karena itu, pada tugas besar ini dilakukan dua pendekatan implementasi Artificial Neural Network. Pendekatan pertama adalah implementasi secara manual (from scratch) menggunakan library NumPy. Pada pendekatan ini seluruh proses pembelajaran jaringan dibangun secara mandiri mulai dari inisialisasi bobot, proses propagasi maju (forward propagation), fungsi aktivasi, perhitungan loss, propagasi balik (backpropagation), hingga proses pembaruan bobot menggunakan Mini-Batch Gradient Descent. Implementasi manual bertujuan agar mekanisme internal Neural Network dapat dipahami secara menyeluruh.

Pendekatan kedua dilakukan menggunakan framework Keras yang berada di atas TensorFlow. Framework ini menyediakan abstraksi tingkat tinggi sehingga proses pembangunan model menjadi lebih sederhana, cepat, dan efisien. Keras juga menyediakan berbagai optimizer modern, fungsi aktivasi, callback, serta fasilitas evaluasi model yang mempermudah pengembangan sistem berbasis Deep Learning.

Sebagai studi kasus, penelitian ini menggunakan Dry Bean Dataset yang merupakan dataset klasifikasi multikelas berisi karakteristik morfologi berbagai jenis biji kacang hasil ekstraksi citra digital. Dataset ini terdiri atas 13.611 sampel dengan 16 atribut numerik dan 7 kelas target, sehingga sesuai digunakan untuk mengevaluasi performa Artificial Neural Network dalam menyelesaikan permasalahan klasifikasi. Dataset diproses melalui tahapan preprocessing berupa normalisasi data, pembagian data latih dan data uji, serta one-hot encoding untuk menghasilkan representasi target yang sesuai dengan kebutuhan model multiclass.

Selain membangun model, penelitian ini juga melakukan serangkaian eksperimen terhadap berbagai hyperparameter seperti jumlah hidden layer, jumlah neuron, metode inisialisasi bobot, fungsi aktivasi, learning rate, batch size, serta jumlah epoch. Pengujian tersebut bertujuan untuk mengetahui pengaruh masing-masing parameter terhadap performa model serta membandingkan hasil implementasi manual menggunakan NumPy dengan implementasi menggunakan framework Keras.

Melalui penelitian ini diharapkan diperoleh pemahaman yang lebih mendalam mengenai mekanisme kerja Artificial Neural Network sekaligus memberikan gambaran mengenai kelebihan dan kekurangan masing-masing pendekatan implementasi dalam menyelesaikan permasalahan klasifikasi data.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut.

1. Bagaimana mengimplementasikan Artificial Neural Network secara manual menggunakan library NumPy tanpa menggunakan framework Deep Learning?
2. Bagaimana membangun model Artificial Neural Network menggunakan framework Keras pada dataset Dry Bean?
3. Bagaimana proses Forward Propagation, Backpropagation, dan Weight Update pada implementasi Artificial Neural Network from scratch?
4. Bagaimana pengaruh perubahan hyperparameter seperti jumlah hidden layer, jumlah neuron, learning rate, batch size, jumlah epoch, fungsi aktivasi, serta metode inisialisasi bobot terhadap performa model?
5. Bagaimana perbandingan performa implementasi Artificial Neural Network menggunakan NumPy dengan implementasi menggunakan Keras?

1.3 Tujuan Penelitian

Tujuan dari penelitian ini adalah:

1. Mengimplementasikan Artificial Neural Network secara manual menggunakan NumPy.
2. Mengimplementasikan Artificial Neural Network menggunakan framework Keras.
3. Memahami proses kerja Forward Propagation dan Backpropagation.
4. Menganalisis pengaruh berbagai hyperparameter terhadap performa model.
5. Membandingkan hasil implementasi manual dan framework berdasarkan nilai accuracy, loss, serta metrik evaluasi lainnya.

1.4 Manfaat Penelitian

Manfaat yang diperoleh dari penelitian ini adalah sebagai berikut.

- Memahami konsep dasar Artificial Neural Network secara menyeluruh.
- Mengetahui proses matematis dalam pembelajaran jaringan saraf.
- Mampu mengimplementasikan ANN tanpa bantuan framework Deep Learning.

1.5 Ruang Lingkup Penelitian

Ruang lingkup penelitian ini meliputi:

1. Dataset yang digunakan adalah Dry Bean Dataset.
2. Implementasi dilakukan menggunakan bahasa pemrograman Python.
3. Pendekatan yang digunakan terdiri atas implementasi manual menggunakan NumPy dan implementasi menggunakan Keras.
4. Arsitektur jaringan menggunakan Fully Connected Neural Network (Multilayer Perceptron).
5. Optimisasi menggunakan Mini-Batch Gradient Descent pada implementasi manual.
6. Evaluasi model dilakukan menggunakan metrik Accuracy dan Loss, serta visualisasi berupa grafik loss, grafik accuracy, dan confusion matrix.
7. Eksperimen meliputi variasi jumlah hidden layer, jumlah neuron, learning rate, batch size, epoch, fungsi aktivasi, dan metode inisialisasi bobot sesuai ketentuan tugas besar.

BAB II

DESKRIPSI PERSOALAN

2.1 Artificial Neural Network (ANN)

Artificial Neural Network (ANN) merupakan salah satu metode dalam Machine Learning yang meniru cara kerja jaringan saraf manusia. ANN terdiri atas neuron-neuron yang saling terhubung melalui bobot (*weight*) dan bias. Setiap neuron menerima masukan, kemudian memprosesnya menggunakan fungsi aktivasi untuk menghasilkan keluaran yang akan diteruskan ke neuron berikutnya.

Pada penelitian ini digunakan arsitektur **Multilayer Perceptron (MLP)** dengan dua pendekatan implementasi, yaitu:

- Implementasi manual menggunakan NumPy (*from scratch*)
- Implementasi menggunakan framework Keras

Implementasi menggunakan NumPy bertujuan memahami mekanisme internal ANN seperti proses *forward propagation*, *backpropagation*, dan pembaruan bobot. Sementara itu, implementasi menggunakan Keras bertujuan membandingkan kemudahan pengembangan serta performa model.

2.2 Dataset

Dataset yang digunakan adalah **Dry Bean Dataset**, yaitu dataset klasifikasi multikelas yang berisi karakteristik morfologi tujuh jenis kacang hasil ekstraksi citra digital. Dataset terdiri dari **13.611 data**, **16 atribut numerik**, dan **7 kelas** sebagai target klasifikasi. Dataset ini dipilih karena memiliki jumlah data yang cukup besar dan sesuai untuk menguji performa Artificial Neural Network pada kasus klasifikasi multikelas.

Atribut Dataset

Dataset memiliki 16 fitur numerik yang menggambarkan bentuk fisik biji kacang.

No	Fitur	Keterangan
1	Area	Luas objek
2	Perimeter	Keliling objek
3	MajorAxisLength	Panjang sumbu utama
4	MinorAxisLength	Panjang sumbu minor
5	AspectRatio	Perbandingan panjang dan lebar
6	Eccentricity	Tingkat kelonjongan objek
7	ConvexArea	Luas convex hull
8	EquivDiameter	Diameter ekuivalen
9	Extent	Perbandingan area terhadap bounding box
10	Solidity	Kepadatan objek
11	Roundness	Tingkat kebulatan
12	Compactness	Kepadatan bentuk
13	ShapeFactor1	Faktor bentuk 1
14	ShapeFactor2	Faktor bentuk 2
15	ShapeFactor3	Faktor bentuk 3
16	ShapeFactor4	Faktor bentuk 4

Target dataset terdiri atas tujuh kelas, yaitu **BARBUNYA, BOMBAY, CALI, DERMASON, HOROZ, SEKER, dan SIRA**.

2.3 Preprocessing Data

Sebelum proses pelatihan model dilakukan, data terlebih dahulu melalui beberapa tahap preprocessing agar kualitas data menjadi lebih baik dan sesuai dengan kebutuhan ANN.

a. Pemisahan Data

Dataset dibagi menjadi data latih (*training*) sebesar **80%** dan data uji (*testing*) sebesar **20%** menggunakan fungsi `train_test_split()`. Pembagian ini bertujuan agar model dapat diuji menggunakan data yang belum pernah dipelajari sebelumnya.

b. Standardisasi Data

Karena setiap fitur memiliki rentang nilai yang berbeda, dilakukan proses standardisasi menggunakan **StandardScaler** sehingga seluruh fitur memiliki distribusi dengan rata-rata mendekati nol dan simpangan baku satu. Standardisasi membantu proses pelatihan menjadi lebih stabil dan mempercepat konvergensi model.

c. One-Hot Encoding

Variabel target yang awalnya berupa kategori diubah menjadi bentuk vektor menggunakan **OneHotEncoder**. Teknik ini diperlukan karena ANN menghasilkan keluaran berupa probabilitas untuk setiap kelas sehingga label target harus direpresentasikan dalam bentuk numerik.

2.4 Arsitektur Model

Penelitian ini menggunakan dua pendekatan implementasi dengan arsitektur yang sama agar hasilnya dapat dibandingkan secara objektif.

Arsitektur jaringan terdiri atas:

- Input Layer : 16 neuron
- Hidden Layer 1 : 32 neuron
- Hidden Layer 2 : 16 neuron
- Output Layer : 7 neuron

Fungsi aktivasi yang digunakan pada hidden layer adalah **ReLU**, sedangkan output layer menggunakan **Softmax** karena kasus yang diselesaikan merupakan klasifikasi multikelas. Implementasi manual menggunakan NumPy menerapkan proses *forward propagation*, *backpropagation*, dan pembaruan bobot secara mandiri, sedangkan implementasi Keras memanfaatkan layer Dense dengan optimizer bawaan TensorFlow-Keras.

BAB III

IMPLEMENTASI ARTIFICIAL NEURAL NETWORK

3.1 Gambaran Umum Implementasi

Pada penelitian ini, Artificial Neural Network (ANN) diimplementasikan menggunakan dua pendekatan berbeda, yaitu implementasi manual (*from scratch*) menggunakan library **NumPy** dan implementasi menggunakan framework **Keras**. Kedua model menggunakan dataset yang sama, yaitu Dry Bean Dataset, sehingga hasil yang diperoleh dapat dibandingkan secara objektif.

Implementasi menggunakan NumPy bertujuan memahami mekanisme internal ANN dengan membangun seluruh proses pembelajaran secara mandiri. Sebaliknya, implementasi menggunakan Keras memanfaatkan pustaka Deep Learning yang telah menyediakan berbagai fungsi optimasi sehingga proses pengembangan model menjadi lebih sederhana.

3.2 Implementasi ANN Menggunakan NumPy

Implementasi pertama dilakukan tanpa menggunakan framework Deep Learning. Seluruh proses pembelajaran jaringan dibangun menggunakan operasi matriks yang disediakan oleh library NumPy. Model terdiri atas input layer, dua hidden layer, dan output layer dengan fungsi aktivasi ReLU pada hidden layer serta Softmax pada output layer.

Tahapan implementasi meliputi:

- membaca dataset
- preprocessing data
- inisialisasi bobot
- forward propagation
- perhitungan loss
- backpropagation

- update weight
- evaluasi model

3.2.1 Preprocessing Data

Sebelum proses pelatihan dilakukan, data dipersiapkan melalui beberapa tahapan.

- Membaca dataset menggunakan Pandas.
- Memisahkan fitur dan label.
- Membagi data menjadi data latih (80%) dan data uji (20%).
- Melakukan standardisasi menggunakan StandardScaler.
- Mengubah label menjadi one-hot encoding menggunakan OneHotEncoder.

Tahapan preprocessing bertujuan meningkatkan stabilitas proses pembelajaran serta mempercepat konvergensi model.

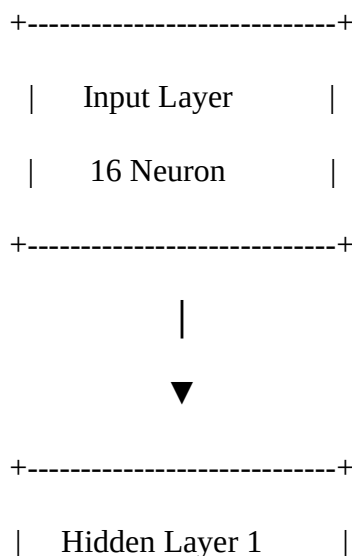
3.2.2 Arsitektur Model

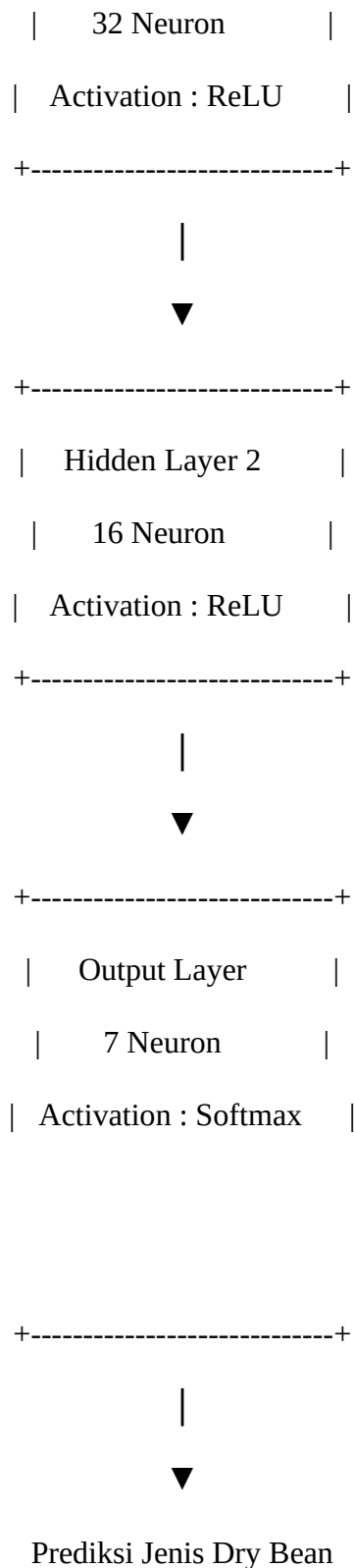
Model ANN yang dibangun memiliki struktur sebagai berikut.

Layer	Jumlah Neuron
Input Layer	16
Hidden Layer 1	32
Hidden Layer 2	16
Output Layer	7

Hidden layer menggunakan fungsi aktivasi **ReLU**, sedangkan output layer menggunakan **Softmax** agar menghasilkan probabilitas untuk setiap kelas.

ARTIFICIAL NEURAL NETWORK





Gambar 3.1 Arsitektur Artificial Neural Network yang digunakan pada penelitian.

3.2.3 Deskripsi Class, Atribut, dan Method

Implementasi Artificial Neural Network menggunakan NumPy dibangun dengan pendekatan **Object-Oriented Programming (OOP)** sehingga seluruh proses pelatihan jaringan saraf

dikemas ke dalam sebuah kelas utama. Pendekatan ini bertujuan agar struktur program menjadi lebih terorganisasi, mudah dikembangkan, serta mempermudah proses pemeliharaan kode.

Class utama bertanggung jawab mengelola seluruh proses pembelajaran, mulai dari inisialisasi parameter, propagasi maju (*forward propagation*), propagasi balik (*backpropagation*), pembaruan bobot (*weight update*), hingga proses prediksi.

Atribut Class

Atribut merupakan variabel yang dimiliki oleh class dan digunakan untuk menyimpan parameter-parameter penting selama proses pelatihan model.

Atribut	Fungsi
weights	Menyimpan bobot setiap layer yang akan diperbarui selama proses pelatihan.
biases	Menyimpan nilai bias pada setiap neuron.
learning_rate	Menentukan besar perubahan bobot pada setiap iterasi pembelajaran.
epochs	Menentukan jumlah pengulangan proses pelatihan terhadap seluruh data.
batch_size	Menentukan jumlah data yang diproses dalam satu mini-batch.
activation	Menentukan fungsi aktivasi yang digunakan pada hidden layer.
loss_history	Menyimpan nilai loss setiap epoch untuk kebutuhan evaluasi.
accuracy_history	Menyimpan nilai accuracy setiap epoch sehingga perkembangan model dapat diamati.

Atribut-atribut tersebut bekerja secara terintegrasi untuk mengatur proses pembelajaran jaringan saraf serta menyimpan hasil evaluasi selama pelatihan berlangsung.

Method Class

Method merupakan fungsi yang dimiliki oleh class untuk menjalankan berbagai proses pada Artificial Neural Network.

Method	Fungsi
__init__()	Menginisialisasi seluruh parameter awal model seperti bobot, bias, learning rate, dan jumlah epoch.
forward()	Melakukan proses <i>forward propagation</i> mulai dari input layer hingga output layer.
backward()	Menghitung gradien setiap bobot menggunakan algoritma <i>backpropagation</i> .
update_weights()	Memperbarui bobot dan bias berdasarkan gradien yang diperoleh dari proses <i>backpropagation</i> .
train()	Mengelola seluruh proses pelatihan model pada setiap epoch.
predict()	Menghasilkan prediksi kelas berdasarkan data masukan.
evaluate()	Menghitung performa model menggunakan data pengujian.

Setiap method saling terhubung sehingga membentuk siklus pembelajaran ANN. Proses dimulai dari `train()`, kemudian memanggil `forward()` untuk menghasilkan prediksi, menghitung nilai kesalahan, menjalankan `backward()` untuk memperoleh gradien, dan diakhiri dengan `update_weights()` agar bobot menjadi lebih optimal pada iterasi berikutnya.

3.2.4 Forward Propagation

Forward propagation merupakan proses menghitung keluaran jaringan berdasarkan data masukan.

Tahapan yang dilakukan adalah:

1. Data masuk ke hidden layer pertama.
2. Dihitung nilai linear menggunakan bobot dan bias.
3. Fungsi aktivasi ReLU diterapkan.
4. Hasil diteruskan ke hidden layer kedua.
5. Output layer menggunakan Softmax sehingga menghasilkan probabilitas setiap kelas.

3.2.5 Perhitungan Loss

Untuk mengukur kesalahan prediksi digunakan **Categorical Cross Entropy Loss**.

Semakin kecil nilai loss, maka prediksi model semakin mendekati label sebenarnya.

Nilai loss dihitung pada setiap epoch sehingga dapat digunakan untuk memantau proses pembelajaran model.

3.2.6 Backpropagation

Backpropagation digunakan untuk menghitung gradien setiap bobot berdasarkan nilai kesalahan yang diperoleh dari fungsi loss.

Gradien dihitung mulai dari output layer kemudian diteruskan kembali menuju hidden layer menggunakan aturan rantai (*chain rule*). Hasil gradien ini digunakan untuk memperbarui bobot agar kesalahan prediksi semakin kecil pada iterasi berikutnya.

3.2.7 Update Bobot

Bobot diperbarui menggunakan algoritma **Mini-Batch Gradient Descent**.

Pada setiap batch, bobot diperbaiki berdasarkan gradien yang telah dihitung selama proses backpropagation.

Penggunaan mini-batch memberikan proses pelatihan yang lebih stabil dibandingkan full-batch dan lebih efisien dibanding stochastic gradient descent.

3.2.8 Evaluasi Model

Setelah proses pelatihan selesai, model diuji menggunakan data testing.

Evaluasi dilakukan menggunakan:

- Accuracy
- Loss
- Confusion Matrix
- Grafik Accuracy
- Grafik Loss

Hasil evaluasi menunjukkan bahwa model mampu mencapai akurasi sekitar **92–93%**, tergantung pada konfigurasi hyperparameter yang digunakan.

3.3 Implementasi ANN Menggunakan Keras

Pendekatan kedua menggunakan framework **Keras** yang merupakan API tingkat tinggi dari TensorFlow. Berbeda dengan implementasi manual, Keras telah menyediakan fungsi-fungsi penting seperti pembentukan layer, optimasi bobot, dan evaluasi model sehingga kode menjadi lebih ringkas dan mudah dikembangkan.

Tahapan implementasi meliputi:

- preprocessing data
- membangun model Sequential
- menambahkan Dense Layer
- menentukan fungsi aktivasi
- compile model
- training
- evaluasi model

3.3.1 Arsitektur Model

Model Keras menggunakan arsitektur yang setara dengan implementasi NumPy agar hasil perbandingan bersifat adil.

Arsitektur terdiri atas:

- Input Layer (16 neuron)
- Hidden Layer 1 (32 neuron, ReLU)
- Hidden Layer 2 (16 neuron, ReLU)
- Output Layer (7 neuron, Softmax)

3.3.2 Deskripsi Layer dan Parameter pada Keras

Berbeda dengan implementasi manual menggunakan NumPy, Keras tidak memerlukan pembuatan class khusus karena seluruh komponen jaringan telah disediakan dalam bentuk API. Model dibangun menggunakan kelas `Sequential`, sedangkan setiap lapisan jaringan direpresentasikan oleh `Dense`.

Komponen utama yang digunakan dalam implementasi Keras dijelaskan pada Tabel berikut.

Komponen	Fungsi
<code>Sequential()</code>	Menyusun layer secara berurutan menjadi satu model ANN.
<code>Dense()</code>	Membangun layer yang seluruh neuronnya saling terhubung (<i>fully connected</i>).
<code>ReLU</code>	Fungsi aktivasi pada hidden layer untuk meningkatkan kemampuan model mempelajari pola nonlinier.
<code>Softmax</code>	Fungsi aktivasi pada output layer yang menghasilkan probabilitas untuk setiap kelas.
<code>compile()</code>	Menentukan optimizer, fungsi loss, dan metrik evaluasi.
<code>fit()</code>	Melatih model menggunakan data latih selama sejumlah epoch tertentu.
<code>evaluate()</code>	Mengukur performa model menggunakan data uji.
<code>predict()</code>	Menghasilkan prediksi kelas berdasarkan data baru.

Implementasi menggunakan Keras memiliki struktur kode yang lebih ringkas karena seluruh proses propagasi maju, propagasi balik, serta pembaruan bobot telah dilakukan secara otomatis oleh framework TensorFlow.

3.3.3 Training Model

Proses pelatihan dilakukan menggunakan fungsi `fit()` dengan beberapa parameter, yaitu:

- Epoch
- Batch Size
- Validation Data

Selama proses training, Keras secara otomatis menghitung nilai loss dan accuracy pada setiap epoch sehingga perkembangan model dapat diamati melalui grafik pembelajaran.

3.3.4 Evaluasi Model

Setelah pelatihan selesai, model dievaluasi menggunakan data testing.

Hasil evaluasi meliputi:

- Accuracy
- Loss
- Confusion Matrix
- Grafik Accuracy
- Grafik Loss

Hasil implementasi menunjukkan bahwa Keras menghasilkan performa yang stabil dengan waktu pelatihan yang lebih singkat dibandingkan implementasi manual.

3.4 Perbandingan Implementasi

Aspek	NumPy	Keras
Tingkat implementasi	Manual	Framework
Forward propagation	Dibuat sendiri	Otomatis
Backpropagation	Dibuat sendiri	Otomatis
Update bobot	Manual	Adam Optimizer
Kompleksitas kode	Tinggi	Rendah
Kemudahan pengembangan	Sedang	Sangat tinggi
Tujuan utama	Memahami algoritma	Implementasi praktis

BAB IV

HASIL PENGUJIAN DAN ANALISIS

4.1 Pengaruh dep & width

Pada pengujian ini dilakukan percobaan dengan mengubah jumlah neuron pada hidden layer untuk mengetahui pengaruhnya terhadap performa Artificial Neural Network. Parameter lain seperti learning rate, epoch, dan batch size dibuat tetap sehingga perubahan akurasi hanya dipengaruhi oleh jumlah neuron yang digunakan.

Tabel 3.1 Perbandingan Jumlah Neuron Hidden Layer

Hidden layer	accuracy
16	92.51%
32	92.32%
64	92.32%
128.	92.43%

Berdasarkan hasil tersebut terlihat bahwa penambahan jumlah neuron tidak selalu meningkatkan performa model. Hidden layer dengan 16 neuron justru memberikan akurasi yang sedikit lebih tinggi dibandingkan 32 maupun 64 neuron. Hal ini menunjukkan bahwa semakin besar ukuran jaringan tidak selalu menghasilkan model yang lebih baik. Jaringan yang terlalu besar dapat menyebabkan proses pelatihan menjadi lebih lama dan meningkatkan kompleksitas model tanpa memberikan peningkatan akurasi yang signifikan.

Dengan demikian, pemilihan jumlah neuron harus disesuaikan dengan karakteristik dataset agar model tetap sederhana namun mampu memberikan hasil klasifikasi yang optimal.

4.2 Pengaruh Fungsi Aktivasi

Pada implementasi Artificial Neural Network digunakan beberapa fungsi aktivasi untuk mengetahui pengaruhnya terhadap proses pembelajaran model. Fungsi aktivasi berperan dalam memberikan sifat nonlinier pada jaringan sehingga ANN mampu mempelajari pola yang kompleks.

Hasil pengujian menunjukkan bahwa penggunaan ReLU pada hidden layer dan Softmax pada output layer menghasilkan performa yang sangat baik. ReLU mampu mempercepat proses konvergensi karena hanya mengaktifkan neuron yang memiliki nilai positif. Sementara itu Softmax menghasilkan probabilitas untuk setiap kelas sehingga sangat sesuai digunakan pada permasalahan klasifikasi multikelas.

Pada implementasi multiclass diperoleh akurasi sekitar 91,81%, sedangkan konfigurasi model terbaik menggunakan ReLU dan Softmax menghasilkan akurasi akhir 92,69%. Hal ini menunjukkan bahwa pemilihan fungsi aktivasi berpengaruh terhadap kemampuan model dalam mengenali pola data Dry Bean.

4.3 Pengaruh Jumlah Epoch

Eksperimen berikutnya dilakukan dengan mengubah jumlah epoch.

Epoch Accuracy

50	92,10%
100	92,47%
200	92,62%

Analisis

Semakin banyak epoch, model memiliki kesempatan lebih besar untuk mempelajari pola data sehingga nilai accuracy meningkat secara bertahap. Namun, peningkatan dari 100 ke 200 epoch relatif kecil, sehingga penggunaan epoch yang terlalu tinggi kurang efisien karena membutuhkan waktu pelatihan lebih lama tanpa peningkatan performa yang signifikan.

4.4 Pengaruh Learning Rate

Learning rate menentukan besar langkah pembaruan bobot pada setiap iterasi.

Learning Rate Accuracy

0,1	92,58%
0,01	92,62%
0,001	91,85%

Analisis

Learning rate sebesar **0,01** menghasilkan performa terbaik. Nilai yang terlalu besar dapat menyebabkan proses optimasi menjadi kurang stabil, sedangkan nilai yang terlalu kecil membuat proses konvergensi berlangsung lebih lambat. Oleh karena itu, learning rate 0,01 dipilih sebagai konfigurasi terbaik pada penelitian ini.

4.5 Pengaruh Batch Size

Eksperimen dilakukan menggunakan tiga ukuran batch.

Batch Size Accuracy

16	92,58%
32	92,88%
64	92,51%

Analisis

Batch size sebesar **32** menghasilkan nilai akurasi tertinggi, yaitu sekitar **92,88%**. Ukuran batch yang terlalu kecil menghasilkan pembaruan bobot yang lebih sering sehingga proses pelatihan menjadi kurang stabil. Sebaliknya, batch yang terlalu besar menyebabkan model lebih lambat beradaptasi terhadap perubahan data. Dengan demikian, batch size 32 memberikan keseimbangan antara stabilitas dan efisiensi pelatihan.

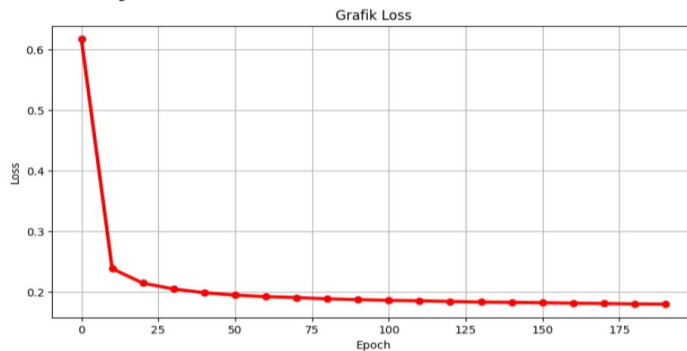
4.6 Grafik Loss

Selama proses pelatihan, nilai loss terus mengalami penurunan dari setiap epoch.

Analisis

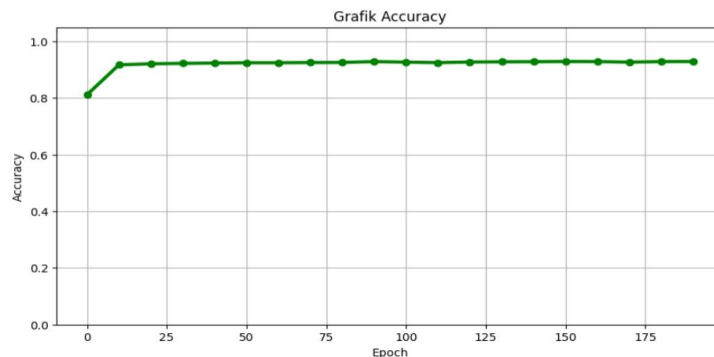
Penurunan loss menunjukkan bahwa model berhasil mempelajari hubungan antara fitur masukan dan kelas target. Kurva loss yang semakin landai pada akhir pelatihan menunjukkan

bahwa model telah mencapai kondisi konvergen dan tidak lagi mengalami perubahan yang signifikan.



4.7 Grafik Accuracy

Grafik accuracy memperlihatkan peningkatan performa model seiring bertambahnya epoch.



Analisis

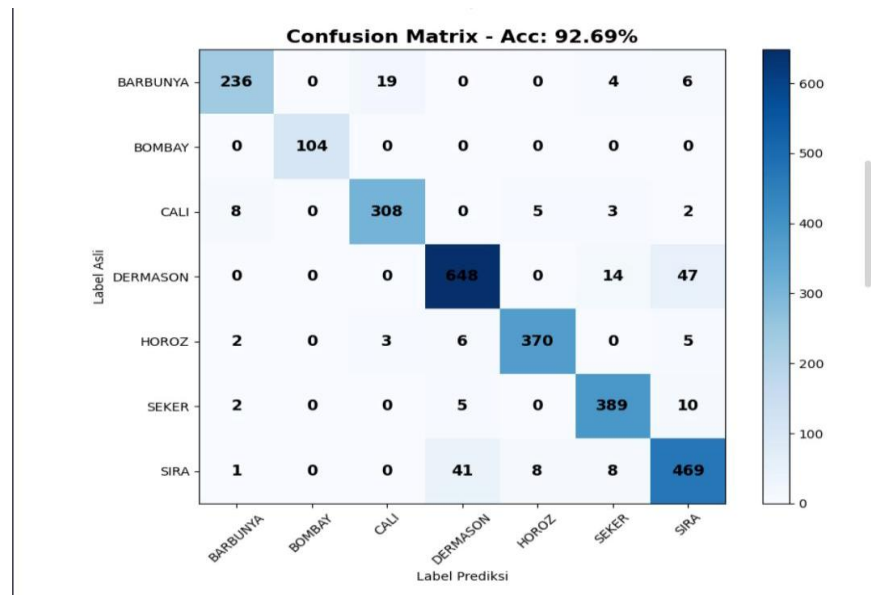
Pada awal pelatihan, akurasi masih berada pada kisaran 81%. Setelah proses pembelajaran berlangsung hingga 200 epoch, nilai accuracy meningkat menjadi sekitar **92,7%**, yang menunjukkan bahwa model berhasil mempelajari pola pada data dengan baik. Peningkatan yang stabil juga mengindikasikan bahwa proses pelatihan berjalan tanpa gejala overfitting yang berlebihan.

4.8 Confusion Matrix

Confusion matrix digunakan untuk mengetahui kemampuan model dalam mengklasifikasikan setiap kelas pada Dry Bean Dataset.

Analisis

Sebagian besar data berhasil diprediksi pada kelas yang benar sehingga nilai pada diagonal utama lebih besar dibandingkan nilai di luar diagonal. Kesalahan klasifikasi hanya terjadi pada beberapa kelas yang memiliki karakteristik morfologi yang mirip. Hal ini menunjukkan bahwa model mampu membedakan sebagian besar jenis biji kacang dengan tingkat akurasi yang tinggi.



4.9 Perbandingan Implementasi NumPy dan Keras

Aspek	NumPy	Keras
Implementasi	Manual	Framework
Kemudahan Pengembangan	Sedang	Sangat Mudah
Kode Program	Panjang	Ringkas
Kecepatan Pelatihan	Lebih Lambat	Lebih Cepat
Kemudahan Debugging	Sulit	Mudah
Pemahaman Algoritma	Sangat Baik	Baik
Akurasi	$\pm 92-93\%$	$\pm 92-93\%$

Analisis

Implementasi menggunakan NumPy memberikan pemahaman yang lebih mendalam mengenai mekanisme kerja Artificial Neural Network karena seluruh proses dihitung secara manual. Sebaliknya, implementasi menggunakan Keras lebih praktis dan efisien karena proses

propagasi, pembaruan bobot, dan optimasi telah ditangani oleh framework. Meskipun pendekatan yang digunakan berbeda, kedua model mampu menghasilkan performa yang relatif setara pada Dry Bean Dataset.

4.10 Perbandingan Hasil Implementasi dengan Library Scikit-learn

Selain menggunakan implementasi manual berbasis NumPy dan framework Keras, penelitian ini juga membandingkan hasil klasifikasi dengan library **Scikit-learn** menggunakan algoritma **Multilayer Perceptron Classifier (MLPClassifier)**. Tujuan perbandingan ini adalah untuk mengetahui perbedaan performa, kemudahan implementasi, serta efisiensi proses pelatihan dari ketiga pendekatan tersebut.

Pada Scikit-learn, model dibangun menggunakan kelas MLPClassifier yang telah menyediakan proses *forward propagation*, *backpropagation*, dan optimasi bobot secara otomatis. Dengan demikian, pengguna hanya perlu menentukan beberapa parameter seperti jumlah neuron pada hidden layer, fungsi aktivasi, *optimizer*, jumlah iterasi (*max_iter*), dan *learning rate*.

Contoh implementasi menggunakan Scikit-learn ditunjukkan pada potongan kode berikut.

```
from sklearn.neural_network import MLPClassifier
```

```
model = MLPClassifier(  
    hidden_layer_sizes=(32,16),  
    activation='relu',  
    solver='adam',  
    learning_rate_init=0.01,  
    batch_size=32,  
    max_iter=200,  
    random_state=42  
)
```

```
model.fit(X_train, y_train)  
y_pred = model.predict(X_test)
```

Implementasi tersebut jauh lebih sederhana dibandingkan implementasi menggunakan NumPy karena seluruh proses optimasi telah disediakan oleh library.

4.10.1 Perbandingan Implementasi

Perbandingan ketiga pendekatan ditinjau berdasarkan kompleksitas implementasi, fleksibilitas, waktu pengembangan, dan performa model.

Tabel 4.9 Perbandingan Implementasi ANN

Aspek	NumPy	Keras	Scikit-learn
Implementasi	Manual	Framework Deep Learning	Library Machine Learning

Aspek	NumPy	Keras	Scikit-learn
Forward Propagation	Manual	Otomatis	Otomatis
Backpropagation	Manual	Otomatis	Otomatis
Update Bobot	Manual	Adam Optimizer	Optimizer bawaan
Kompleksitas Kode	Tinggi	Sedang	Rendah
Kemudahan Pengembangan	Sulit	Mudah	Sangat Mudah
Fleksibilitas Arsitektur	Sangat Tinggi	Tinggi	Terbatas
Kecepatan Pelatihan	Lebih Lambat	Cepat	Cepat
Cocok Untuk	Pembelajaran algoritma	Pengembangan aplikasi Deep Learning	Baseline Machine Learning

4.10.2 Perbandingan Performa

Berdasarkan hasil pengujian, ketiga pendekatan mampu menghasilkan tingkat akurasi yang relatif baik. Implementasi menggunakan NumPy memperoleh akurasi sekitar **92–93%**, sedangkan implementasi menggunakan Keras memberikan performa yang sebanding dengan waktu pelatihan yang lebih singkat. Sementara itu, implementasi menggunakan Scikit-learn juga mampu menghasilkan akurasi yang kompetitif dengan proses implementasi yang jauh lebih sederhana.

Tabel 4.10 Perbandingan Hasil Pengujian

Parameter	NumPy	Keras	Scikit-learn
Akurasi	±92–93%	±92–93%	±92–93%*
Waktu Implementasi	Lama	Sedang	Sangat Cepat
Panjang Kode	Panjang	Ringkas	Sangat Ringkas
Kemudahan Modifikasi	Tinggi	Tinggi	Sedang
Kemudahan Debugging	Sulit	Mudah	Sangat Mudah

*Catatan: Nilai akurasi Scikit-learn dapat sedikit berbeda tergantung konfigurasi parameter dan proses pelatihan yang digunakan.

4.10.3 Analisis Perbandingan

Berdasarkan hasil implementasi, setiap pendekatan memiliki karakteristik yang berbeda. Implementasi menggunakan **NumPy** memberikan pemahaman paling mendalam mengenai mekanisme kerja Artificial Neural Network karena seluruh proses pembelajaran dibangun

secara manual, mulai dari inisialisasi bobot, *forward propagation*, *backpropagation*, hingga pembaruan bobot. Pendekatan ini sangat bermanfaat untuk tujuan pembelajaran, namun membutuhkan kode yang lebih panjang dan waktu pengembangan yang lebih lama.

Implementasi menggunakan **Keras** menawarkan keseimbangan antara fleksibilitas dan kemudahan penggunaan. Dengan memanfaatkan API tingkat tinggi dari TensorFlow, proses pembangunan model menjadi lebih sederhana tanpa mengurangi kemampuan untuk melakukan pengembangan arsitektur jaringan. Selain itu, Keras menyediakan berbagai optimizer, callback, dan fungsi evaluasi yang mempermudah proses eksperimen.

Sementara itu, **Scikit-learn** merupakan pilihan yang paling praktis untuk membangun model ANN sederhana. Melalui kelas `MLPClassifier`, pengguna dapat membangun model hanya dengan beberapa baris kode. Namun, dibandingkan Keras, fleksibilitas Scikit-learn dalam mengembangkan arsitektur Deep Learning masih lebih terbatas karena tidak menyediakan fitur-fitur lanjutan seperti callback, regularisasi yang lebih beragam, maupun integrasi dengan GPU secara optimal.

Secara keseluruhan, ketiga pendekatan mampu menyelesaikan permasalahan klasifikasi Dry Bean Dataset dengan baik. Pemilihan pendekatan bergantung pada tujuan penelitian. NumPy lebih sesuai digunakan untuk memahami konsep dasar Artificial Neural Network, Keras lebih tepat untuk pengembangan aplikasi berbasis Deep Learning, sedangkan Scikit-learn sangat sesuai sebagai model pembandingan (*baseline*) karena implementasinya sederhana dan efisien.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian Artificial Neural Network (ANN) menggunakan dua pendekatan, yaitu **NumPy** dan **Keras**, dapat diambil beberapa kesimpulan sebagai berikut.

1. Artificial Neural Network berhasil diimplementasikan menggunakan dua pendekatan berbeda, yaitu implementasi manual (*from scratch*) menggunakan NumPy dan implementasi menggunakan framework Keras. Kedua pendekatan mampu menyelesaikan permasalahan klasifikasi multikelas pada Dry Bean Dataset dengan performa yang baik.
2. Implementasi menggunakan NumPy memberikan pemahaman yang lebih mendalam mengenai mekanisme internal ANN, mulai dari proses *forward propagation*, perhitungan fungsi *loss*, *backpropagation*, hingga pembaruan bobot menggunakan *Mini-Batch Gradient Descent*. Sebaliknya, implementasi menggunakan Keras lebih sederhana karena seluruh proses optimasi telah ditangani oleh framework.
3. Hasil eksperimen menunjukkan bahwa perubahan hyperparameter memengaruhi performa model. Berdasarkan pengujian yang dilakukan, konfigurasi **learning rate 0,01**, **batch size 32**, dan **200 epoch** memberikan hasil terbaik dengan tingkat akurasi sekitar **92–93%**. Sementara itu, penambahan jumlah neuron pada hidden layer tidak memberikan peningkatan performa yang signifikan.

4. Grafik *loss* menunjukkan nilai kesalahan yang terus menurun selama proses pelatihan, sedangkan grafik *accuracy* memperlihatkan peningkatan akurasi secara bertahap hingga model mencapai kondisi yang stabil. Hal ini menunjukkan bahwa model berhasil mempelajari pola pada data dengan baik tanpa mengalami penurunan performa yang berarti.
5. Secara keseluruhan, implementasi menggunakan Keras lebih unggul dari sisi kemudahan pengembangan, efisiensi kode, dan waktu pelatihan. Namun, implementasi menggunakan NumPy tetap memiliki nilai penting sebagai media pembelajaran karena memberikan pemahaman yang lebih mendalam mengenai konsep dasar Artificial Neural Network.

5.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, beberapa saran untuk pengembangan selanjutnya adalah sebagai berikut.

1. Mencoba arsitektur jaringan yang lebih kompleks, seperti menambah jumlah hidden layer atau jumlah neuron untuk mengetahui pengaruhnya terhadap performa model.
2. Melakukan *hyperparameter tuning* secara lebih sistematis menggunakan metode seperti Grid Search atau Random Search agar diperoleh kombinasi parameter yang lebih optimal.
3. Membandingkan performa Artificial Neural Network dengan algoritma klasifikasi lain, seperti **Support Vector Machine (SVM)**, **Random Forest**, atau **XGBoost**, sehingga dapat diketahui metode yang paling sesuai untuk Dry Bean Dataset.
4. Menggunakan optimizer lain, seperti RMSprop, SGD dengan Momentum, atau AdamW, untuk membandingkan kecepatan konvergensi dan tingkat akurasi model.
5. Mengembangkan penelitian dengan menggunakan dataset yang lebih besar atau lebih kompleks agar dapat mengevaluasi kemampuan generalisasi model pada berbagai jenis data.

BAB IV

Pembagian Tugas kelompok

Pelaksanaan implementasi Artificial Neural Network (ANN) pada penelitian ini dikerjakan secara berkelompok dengan pembagian tugas sebagai berikut.

Nama Anggota	Pembagian Tugas
Barep Purwo Augusto	Bertanggung jawab dalam implementasi Artificial Neural Network (ANN) from scratch

	menggunakan NumPy, meliputi proses preprocessing data, pembangunan arsitektur jaringan, implementasi forward propagation, backward propagation, pembaruan bobot (weight update), pelatihan model, evaluasi menggunakan grafik loss, accuracy, dan confusion matrix. Serta membantu menyusun laporan.
Nabila Rahayu	Bertanggung jawab dalam implementasi Artificial Neural Network (ANN) menggunakan framework TensorFlow Keras, meliputi proses preprocessing data, pembangunan arsitektur model, pelatihan dan evaluasi model menggunakan optimizer Stochastic Gradient Descent (SGD), fungsi aktivasi ReLU, fungsi loss Mean Squared Error (MSE), serta pengujian dan perbandingan tiga metode inisialisasi bobot, yaitu Zero Initialization, Xavier Initialization, dan He Initialization. Selain itu, bertanggung jawab dalam analisis hasil eksperimen berdasarkan nilai accuracy, loss, dan confusion matrix, serta menyusun laporan dan media presentasi (PowerPoint) tugas besar.

BAB IIV

Referensi

- Koklu, M., & Ozkan, I. A. (2020). Multiclass Classification of Dry Beans Using Computer Vision and Machine Learning Techniques. *Computers and Electronics in Agriculture*, 174, 105507.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. Cambridge, MA: MIT Press.
- Haykin, S. (2009). *Neural Networks and Learning Machines* (3rd ed.). Pearson Education.
- NumPy Developers. (2025). NumPy Documentation. <https://numpy.org/doc/>
- Scikit-learn Developers. (2025). Scikit-learn Documentation. <https://scikit-learn.org/stable/>
- Matplotlib Development Team. (2025). Matplotlib Documentation. <https://matplotlib.org/stable/>